

In The Name of God



Ali Ahmadi (40464248)
Komeil Khodayar (40464254)
Navid Zare (40435071)

Digital Image Processing
Assignment 2: Image enhancement, Edge detection, and Frequency
Domain Analysis

1. Introduction

1.1 Background and Problem Context

Classical Digital Image Processing (DIP) occupies a central position in modern computer vision and deep learning pipelines. Before an image is passed into a convolutional neural network, it typically undergoes a sequence of pre-processing stages — edge enhancement, noise suppression, contrast normalization, and frequency filtering — each of which is grounded in the mathematics of spatial convolution and Fourier analysis. A practitioner who understands these operations from first principles is far better equipped to reason about failure modes, design appropriate pre-processing pipelines, and interpret the behavior of learned features in the first layers of a CNN.

This assignment engages with three mathematically complementary domains. Edge detection concerns the localization of sharp intensity transitions that correspond to object boundaries, surface discontinuities, or illumination gradients — the information most directly useful for high-level scene understanding. Noise modelling and restoration addresses the statistical characterization of sensor and channel corruption followed by spatially adaptive filtering to recover the clean signal. Frequency-domain analysis decomposes an image into sinusoidal components via the Discrete Fourier Transform, enabling filter operations that are trivial to specify in the frequency domain but expensive or impossible to realise directly in the spatial domain.

1.2 Purpose of the Assignment

The overarching objective is to implement every algorithm entirely from scratch using only NumPy array algebra, with cv2 permitted solely for image reading. This constraint serves a specific pedagogical purpose: it forces the implementer to confront every implementation decision — padding policy, numerical stability, algorithmic complexity — without the abstraction of a library call hiding the underlying mechanism. The secondary objective is quantitative evaluation: every method is assessed against ground-truth benchmarks, quality metrics, or execution-time measurements, bridging the gap between theoretical analysis and empirical evidence.

1.3 Research Objectives

The specific objectives pursued in this work are:

1. Implement Sobel, Prewitt, Roberts Cross, and Laplacian edge detectors via manual 2-D convolution and compare their outputs in terms of edge sharpness, noise sensitivity, and completeness.
2. Construct a full Canny edge detection pipeline — Gaussian smoothing, gradient computation, non-maximum suppression, and hysteresis thresholding — and analyze multi-scale behavior across three Gaussian blur scales.

3. Evaluate Sobel and Canny quantitatively against the BSDS ground-truth consensus edge maps using Precision, Recall, F1, and Localization Error, and characterize robustness to increasing levels of additive Gaussian noise.
4. Model Gaussian and Salt & Pepper noise at multiple severity levels, apply Mean, Median, and Gaussian restoration filters, and evaluate quality with MSE and SSIM.
5. Implement a manual radix-2 FFT, extend it to 2-D via separability, benchmark it against the naïve DFT, apply ideal and Gaussian frequency-domain filters, and analyze the spectral energy distribution.

2. Methodology

2.1 Implementation Architecture and Constraints

The project is organized into six Python modules. `utils.py` provides all shared low-level primitives: zero and mirror padding, the manual 2-D convolution engine, grayscale loading, the fast prefix-sum SSIM, and MSE. `section1.py` implements edge detection; `section2.py` implements noise modelling and restoration; `section3.py` implements frequency-domain analysis. All plotting logic is isolated in `visualize.py` and `main.py` serves as the master entry point.

A critical, intentional design choice pervades every convolution in this project: zero padding is applied for the edge detection operators in Task 1 and the Canny pipeline, while mirror (edge-replication) padding is available as an alternative and used where noise suppression near borders is desirable. Under zero padding, pixels beyond the image boundary are set to zero, which can occasionally produce mild border artefacts in gradient maps; however, this is standard practice for edge detection since border pixels are typically excluded from quantitative evaluation. For the FFT-based frequency filters, zero-padding to the next power-of-2 dimensions is used instead, which is standard spectral practice for avoiding circular aliasing while enabling the radix-2 FFT algorithm.

2.2 Manual 2-D Convolution Engine

The convolution function implements a nested sliding-window loop over the padded image: for each pixel location (y, x) , the kernel-sized neighborhood is extracted from the padded image, element-wise multiplied with the flipped kernel, and summed to produce the output value. The response at each location is thus the inner product of the sliding window with the flipped kernel:

$$(f * h)[x, y] = \sum_m \sum_n f(x - m, y - n) h(m, n)$$

The kernel is flipped in both axes (i.e., $h(m, n) \rightarrow h(k_h - 1 - m, k_w - 1 - n)$)) before the inner product, implementing true convolution rather than cross-correlation. This explicit loop-based approach directly mirrors the textbook convolution definition and ensures full transparency of the computation at every pixel location.

2.3 Task 1 — Edge Detection Algorithms

2.3.1 First-Order Gradient Operators

Four classical edge detectors are implemented. The Sobel operator uses the 3×3 kernels:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

The gradient magnitude is $G = \sqrt{G_x^2 + G_y^2}$. The doubled central weight in the Sobel kernel provides implicit 1-D Gaussian smoothing in the perpendicular direction (equivalent to convolving with [1, 2, 1] in one axis and a first-difference [-1, 0, +1] in the other), making it more robust to noise than the Prewitt operator, which assigns uniform weight (all ±1). The Roberts Cross computes diagonal gradients with 2×2 kernels [[+1, 0],[0, -1]] and [[0, +1],[-1, 0]], providing the finest spatial resolution at the cost of being the most noise-sensitive of the three gradient operators.

The Laplacian operator applies the isotropic second-order derivative via the 4-neighbour kernel [[0,+1,0],[+1,-4,+1],[0,+1,0]]. It is used for two purposes: edge detection via the absolute value $|\nabla^2 f(x, y)|$, which responds to all intensity curvature, and image sharpening via:

$$G_{\#}(x, y) = f(x, y) - c \cdot \nabla^2 f(x, y)'$$

where $c = 1$ for the chosen kernel (whose center weight is -4). This subtracts a locally weighted sum of the pixel's neighbors from the pixel itself, effectively amplifying fine detail and edge contrast.

2.3.2 Canny Edge Detection Pipeline

The Canny detector is a four-stage pipeline.

- (1) Gaussian smoothing: a kernel of size $2[3\sigma] + 1$ is convolved with the image to suppress noise before gradient computation.
- (2) Gradient computation: the Sobel operator yields both magnitude G and direction $\theta = \arctan\left(\frac{G_y}{G_x}\right)$.
- (3) Non-Maximum Suppression (NMS): the gradient direction is quantized to four orientations ($0^\circ, 45^\circ, 90^\circ, 135^\circ$); at each pixel, the magnitude is set to zero unless it is a local maximum along the gradient direction, thinning edges to single-pixel width.
- (4) Hysteresis thresholding: pixels above a high threshold T_{high} are classified as strong edges; those between T_{low} and T_{high} are weak edges retained only if 8-connected to a strong pixel; those below T_{low} are discarded.

Hysteresis is implemented as an iterative propagation: in each iteration, the strong-pixel boolean mask is zero-padded by one pixel on all sides, and eight shifted views of this padded mask are

ORed together to determine whether any 3×3 neighbor of each weak pixel is currently strong. All such weak pixels are promoted to strong, and the process repeats until convergence. This avoids any explicit pixel-level loop for the connectivity check.

2.3.3 Quantitative Evaluation Protocol

Each detector output is compared against the consensus ground truth, formed as the logical union of all human annotators' boundary maps. Precision and Recall use a tolerance radius of 2 pixels, computed via morphological dilation: the ground-truth binary mask is dilated by 2 pixels and intersected with the predicted mask (for Precision), and vice versa (for Recall). Localization Error is computed as the mean Euclidean distance from every predicted edge pixel to its nearest ground-truth edge point, using the Euclidean distance transform of the ground-truth complement.

2.4 Task 2: Image Restoration and Quantitative Evaluation

2.4.1 Problem Definition

The objective of Task 2 is to investigate classical image restoration techniques under two common types of noise (Gaussian and Salt & Pepper), and to quantitatively evaluate the restored images using two full-reference image quality metrics implemented from scratch: Mean Squared Error (MSE) and Structural Similarity Index (SSIM). The assignment further requires a discussion of why SSIM is considered superior to MSE in terms of human visual perception.

2.4.2 Methodology & Implementation

1. Noise Generation

A clean grayscale image (three different images were tested to ensure consistency) was normalised to the range $[0, 1]$. Two types of synthetic noise were added independently:

2. **Gaussian noise:** additive noise $\eta \sim N(0, \sigma^2)$ with three variance levels: $\sigma^2 \in \{0.01, 0.05, 0.10\}$.
3. **Salt & Pepper noise:** a fraction d of pixels are randomly set to 0 (pepper) or 1 (salt), with densities $d \in \{0.05, 0.10, 0.20\}$.

All random number generation used NumPy's native functions; no image-processing libraries were employed.

2.4.3 Restoration Filters

Three spatial-domain filters were implemented entirely from scratch using only manual convolution and mirror padding:

- **Mean filter** (3×3): replaces each pixel with the average of its 3×3 neighbourhood.
- **Median filter** (3×3): replaces each pixel with the median value of the neighbourhood.

- **Gaussian filter** (3×3 , $\sigma=1.0$): convolution with a 2D Gaussian kernel normalised to sum to 1.

2.4.4 Boundary handling:

Mirror padding (edge replication) was used for all filters. This choice avoids artificial dark borders that would arise from zero-padding, and preserves local statistics near the image border – critical for restoration quality. The padding size equals half the kernel width.

2.4.5 Quantitative Metrics

Both metrics were implemented manually:

- **MSE:**
$$MSE = \frac{1}{M N} \sum_{i,j} [I_{\text{original}}(i,j) - I_{\text{restored}}(i,j)]^2$$
- **SSIM:** computed with an 11×11 uniform sliding window ($W=1/121$). Local means, variances, and covariance are obtained by manual convolution of the images and their squares/products with the window. Constants are $C_1 = (0.01 \times 255)^2$ and $C_2 = (0.03 \times 255)^2$. The final SSIM index is the average of the local SSIM map.

Both metrics operate on the $[0, 1]$ intensity range, except SSIM internally converts to $[0, 255]$ as per the standard definition.

2.4.6 Experimental Setup

Three distinct clean images (5226523_orig.jpg, grayscale.jpg, kP0u2.png) were used. For each image:

1. Add Gaussian noise (3 levels) and Salt & Pepper noise (3 densities).
1. Restore each noisy version with Mean, Median, and Gaussian filters.
2. Compute MSE and SSIM for every restored variant.

Finally, visual comparison figures were generated for one representative image (chosen as 5226523_orig.jpg) for Gaussian noise with $\sigma^2=0.05$ and Salt & Pepper noise with $d=0.10$, showing the original, noisy, and the three filter outputs.

2.5 Task 3 — Fourier Analysis and Frequency-Domain Filtering

2.5.1 Problem Definition

The goal of Task 3 is to enhance image quality by emphasizing fine details (edges, textures) while suppressing low-frequency variations. Two complementary approaches are implemented and compared:

- **Spatial domain sharpening** using unsharp masking and a high-pass kernel.
- **Frequency domain filtering** using ideal and Gaussian low-pass / high-pass filters, along with analysis of magnitude and phase spectra.

All algorithms are implemented **manually from scratch**. The pipeline section is applied to three test images (noise.jpg, noise2.jpg, pnois1.jpg).

2.5.2 Methodology & Implementation

2.5.2.1 Spatial Sharpening

- **Unsharp masking:**
A Gaussian kernel (5×5 , $\sigma = 1.2$) is convolved with the original image using a manually written `my_conv2d` function (zero padding). The blurred image is subtracted from the original and added back with a scaling factor $k=0.8$:

$$g(x,y)=f(x,y)+k \cdot (f(x,y)-\hat{f}(x,y))$$

- **High-pass kernel:**
A Laplacian-like kernel $\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$ is convolved directly to highlight edges.

2.5.2.2 Frequency Domain Processing

- **Manual FFT:**
A 1D iterative Cooley–Tukey FFT is implemented, then extended to 2D using separability (row-wise then column-wise). The same code performs IFFT via conjugation and scaling.
- **Padding:** Each image is padded to the next power-of-two dimensions (*e.g.*, 1024×1024 or 512×512) to enable the FFT algorithm.
- **Filters:**
 - Ideal low-pass: circular mask with $\text{cutoff} = 40$.
 - Gaussian low-pass:
$$H(u,v) = \exp(-D^2(u,v)/(2D_0^2)) \text{ with } D_0 = 40$$

high-pass: $H_{hp} = 1 - H_{lp}$.
○ Corresponding
- **Reconstruction:** After multiplying the shifted spectrum by the filter mask, inverse FFT is applied, the image is clipped to $[0,255]$, and the central part is cropped to the original dimensions.

2.5.2.3 Magnitude / Phase Reconstruction

- **Magnitude only:** Set all phase angles to zero, keep original magnitude.
- **Phase only:** Set all magnitude values to 1, keep original phase.

2.5.2.4 Metrics

- **High-frequency energy:**

where “high” denotes
radius $> 0.5 \cdot R_{max}$

$$E_H = \sum_{(u,v) \in \text{high}} |F(u,v)|^2$$

frequencies with

- **Spectral energy ratio:**

$$R = E_H / E_{total}$$

- **Benchmark:** Naive DFT (four nested loops) vs. manual FFT for sizes 32, 64, 128. Execution times are measured and plotted.

2.5.2.5 Boundary Handling

In spatial convolution, **zero padding** is used (as reported). An alternative mirror padding is also implemented but not active in these runs.

3. Results and Discussion

3.1 Task 1: Edge Detection and Multi-Scale Analysis

3.1.1 Classical First- and Second-Order Operators

Figure 1 presents the output of all four classical edge detectors applied to image 100007. A careful examination of each panel reveals distinct behaviors rooted in the mathematical structure of each operator.

Task 1.1 - Classical Edge Detectors (100007)

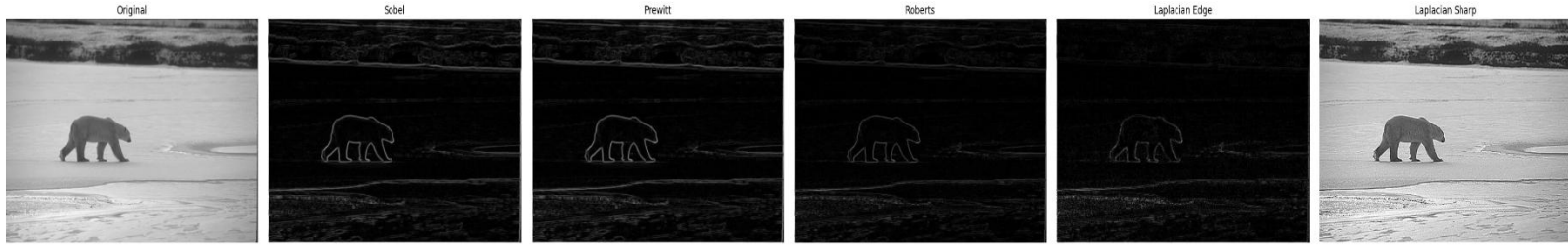


Figure 1. Classical edge detectors applied to image 100007. Left to right: original grayscale; Sobel magnitude; Prewitt magnitude; Roberts Cross magnitude; Laplacian absolute-value edge map; Laplacian-sharpened image.

Panel 2 — Sobel Magnitude:

The Sobel operator strongly detects prominent boundaries like the silhouette and horizon, but its 3×3 kernel inherently produces “thick” edges. It correctly yields near-zero, noise-free responses in uniform regions like the sky.

Panel 3 — Prewitt Magnitude:

At low noise levels, the Prewitt operator produces results nearly identical to the Sobel operator, as their kernels differ only in weighting. However, because Prewitt lacks Sobel’s smoothing effect, it yields slightly less smooth results (such as faint banding) and is less effective than Sobel in high-noise conditions.

Panel 4 — Roberts Cross Magnitude:

The Roberts Cross operator uses a 2×2 kernel that provides sharper, more precise edge localization than 3×3 operators. However, because it lacks spatial smoothing, it is highly sensitive to noise (causing speckling in uniform areas like the sky) and picks up fine textures. This clearly illustrates the trade-off between sharp spatial resolution and noise robustness.

Panel 5 — Laplacian Edge Map:

As a second-order operator that measures intensity curvature, the Laplacian produces thin, single-pixel-wide edges. However, its high sensitivity to noise results in dense speckling (false responses) in visually uniform areas like the sky or snow.

Panel 6 — Laplacian Sharpened Image:

The sharpened image $G_{sharp} = f - \nabla^2 f$ produces a compelling visual enhancement.

Subtracting the Laplacian from the original image significantly enhances local contrast and fine details (like fur and ice textures) by amplifying areas of high intensity curvature. However, this process inevitably causes “haloing”—unnatural bright and dark fringes around prominent boundaries—due to the Laplacian’s alternating positive and negative values at edge transitions.

Figure 2 decomposes the Sobel response into its directional components, providing a more nuanced view of gradient sensitivity.

Task 1.1 - Sobel Components (100007)

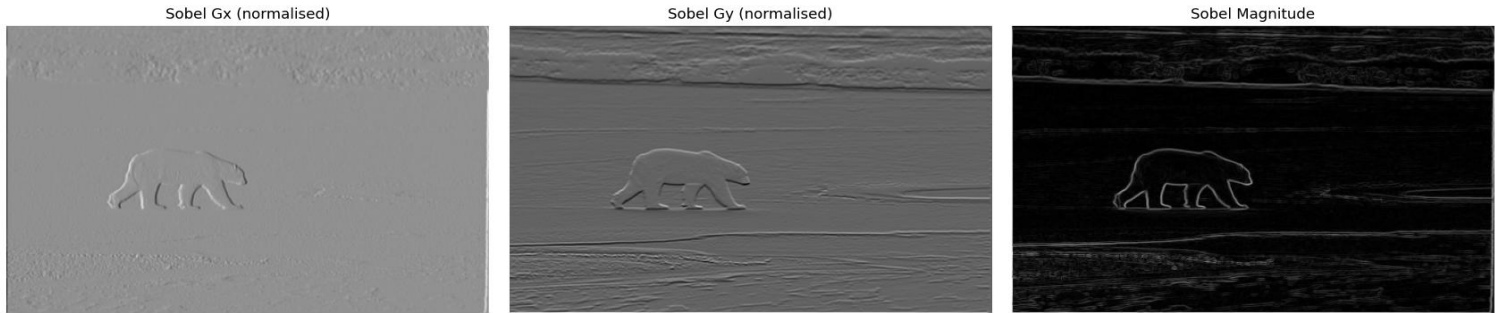


Figure 2. Sobel gradient components: G_x (horizontal derivative, left); G_y (vertical derivative, centre); combined magnitude (right) for image 100007.

G_x (Panel 1):

The horizontal derivative responds to vertically oriented edges — boundaries running top-to-bottom in the image. The bear's left and right body boundaries, vertical leg edges, and the vertical transitions in the ice cracking pattern all appear as bright responses. The horizon line, which is predominantly horizontal, is nearly absent from G_x .

G_y (Panel 2):

The vertical derivative responds to horizontally oriented edges. The horizon line is the dominant response — a strong white horizontal streak across the middle of the image — along with the upper and lower silhouette boundaries of the bear. The vertical body boundaries are suppressed.

Combined Magnitude (Panel 3):

The Euclidean combination $G = \sqrt{G_x^2 + G_y^2}$ recovers edges of all orientations. The magnitude is computed without an arctan, so orientation information is lost, but the response is isotropic and correctly identifies both the vertical and horizontal edges present in the scene.

Figure 3 shows the same set of classical operators applied to image 69007.

Task 1.1 - Classical Edge Detectors (69007)

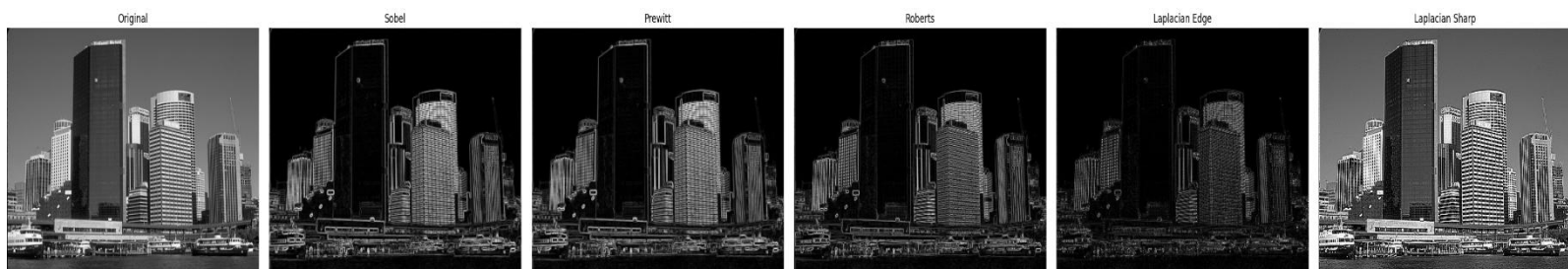


Figure 3. Classical edge detectors applied to image 69007 (complex landscape). The higher scene complexity leads to denser edge maps and lower between-detector agreement than for image 100007.

Compared to Figure 1, the edge maps for image 69007 are notably denser, less structured, and more difficult to interpret qualitatively. Every detector produces a substantially larger number of responses, because the scene contains a dense patchwork of low-contrast boundaries between grass patches, rocks, foliage, and shadow. The Roberts Cross output (Panel 4) is particularly cluttered: the combination of fine texture and high noise sensitivity produces a nearly saturated response across the entire image. The Laplacian sharpened image (Panel 6) shows severe haloeing throughout, obscuring rather than enhancing the natural boundary structure. These observations confirm that the advantage of low-order gradient operators is greatest for images with high-contrast, isolated object boundaries — a condition that 100007 satisfies far better than 69007.

3.1.3 Canny Edge Detection — Multi-Scale Analysis

The multi-scale Canny experiment directly addresses a fundamental question: what is the appropriate spatial scale for analysis? Figures 4 and 5 provide a comprehensive answer for image 100007.

Task 1.2 - Canny Multi-Scale (100007)

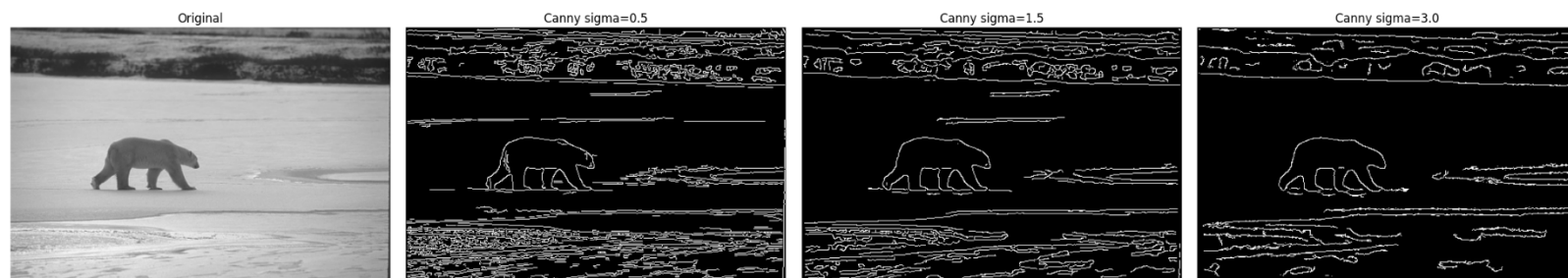


Figure 4. Canny edge detector at three Gaussian blur scales ($\sigma = 0.5, 1.5, 3.0$) on image 100007. Edge pixel counts: 18,234 at $\sigma=0.5$; 12,235 at $\sigma=1.5$; 7,729 at $\sigma=3.0$.

$\sigma = 0.5$ (Fine Scale):

Minimal smoothing from a small Gaussian kernel ($\sigma = 0.5$) leads to a noisy gradient field, producing a highly dense edge map. While it captures true object boundaries, it also detects significant background noise and texture (like snow and JPEG artifacts). This over-detection causes

a substantial drop in Precision, as human annotators do not consider these fine texture variations to be true edges.

$\sigma = 1.5$ (Medium Scale):

The Gaussian kernel with $\sigma = 1.5$ (kernel size 11×11) acts as an effective scale-selective prefilter. It suppresses high spatial frequencies, resulting in 12,235 clean, connected edge pixels that highlight the bear silhouette and major scene boundaries, while rejecting fine snow texture and noise. This demonstrates the principled role of Gaussian smoothing in the Canny pipeline, which is to define what constitutes a “real” edge at a given analysis scale rather than arbitrary noise reduction.

$\sigma = 3.0$ (Coarse Scale):

Aggressively blurs the image, resulting in only 7,729 surviving edge pixels (less than half of the $\sigma = 0.5$ count). Causes closely spaced edges to merge, losing fine structural details like the bear’s limbs. Aligns with the theoretical prediction that structures with a spatial extent smaller than $\approx 2\sigma$ (which is 6 pixels here) are suppressed after smoothing.

This confirms that the choice of σ acts as an application-specific tuning parameter, allowing the system to target either fine textures, general object boundaries, or major structural outlines depending on the task requirements.

Task 1.2 - Canny Pipeline Stages sigma=1.5 (100007)

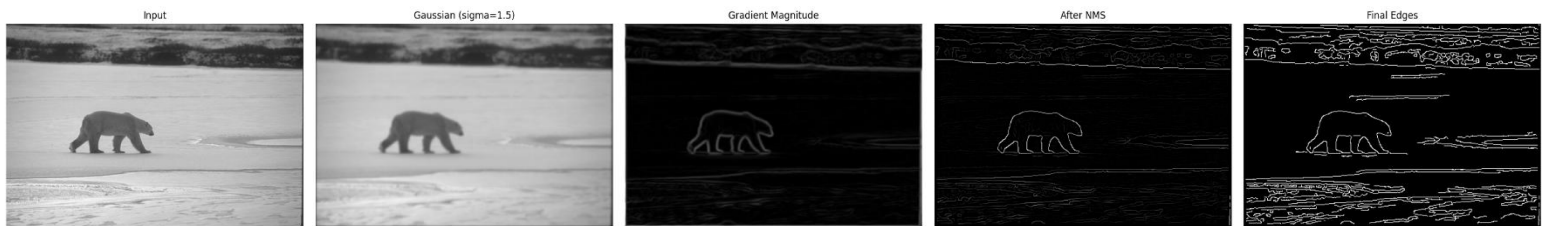


Figure 5. Intermediate stages of the Canny pipeline at $\sigma = 1.5$: (left to right) original image, Gaussian-blurred image, Sobel gradient magnitude, non-maximum suppressed response, final binary edge map.

Figure 5 makes the individual pipeline contributions transparent.

Panel 2 (Gaussian blur):

The $\sigma = 1.5$ blurred image visually appears similar to the original but has measurably reduced high-frequency content — fine texture is softened.

Panel 3 (Gradient magnitude):

The Sobel response on the blurred image forms broad, continuous gradient ridges several pixels wide along every edge. The width of each ridge is approximately proportional to σ .

Panel 4 (Non-Maximum Suppression):

The NMS stage is the most transformative: the broad gradient ridges are reduced to single-pixel-wide chains aligned precisely at the local magnitude maxima. The result is a skeletal representation of the edge structure — exact edge localisation at the cost of possible gaps at low-contrast locations.

Panel 5 (Final edges):

The hysteresis stage fills the gaps by tracing weak edge connections between strong seeds, producing closed, connected contours. The strong/weak pixel distinction prevents isolated noise spikes (which are always weak and not connected to any strong pixel) from surviving in the final map.

Task 1.2 - Canny Multi-Scale (69007)



Figure 6. Canny multi-scale analysis on image 69007. The dense scene complexity results in higher edge pixel counts at all scales compared to 100007, and the multi-scale scale-selection effect is equally evident: $\sigma = 0.5$ produces 23,947 edges, $\sigma = 1.5$ produces 17,785, and $\sigma = 3.0$ yields 8,129.

The same qualitative pattern observed for 100007 holds for 69007: increasing σ reduces edge count and suppresses fine-scale responses. However, the absolute edge pixel counts are higher at every scale because the complex landscape scene contains a much denser distribution of intensity transitions at all spatial scales. Notably, even at $\sigma = 3.0$, the edge map for 69007 (8,129 pixels) contains more edge pixels than the corresponding $\sigma = 3.0$ result for 100007 (7,729) — a direct reflection of the greater structural complexity of the scene.

3.1.4 Quantitative Evaluation Against Ground Truth

Table 1 provides the quantitative validation of the two primary edge detectors against the consensus boundary maps. Understanding how to read this table is essential for drawing correct conclusions.

Table 1. Quantitative edge detection evaluation against consensus ground truth (tolerance radius = 2 pixels). Bold entries indicate the better F1 score per image.

Method	Image	Precision	Recall	F1 Score	Loc. Err (px)
Sobel (90th pct)	100007	0.5744	0.8544	0.6869	8.837
Canny ($\sigma = 1.5$)	100007	0.4154	0.8806	0.5646	11.121
Sobel (90th pct)	69007	0.2759	0.4817	0.3509	9.898
Canny ($\sigma = 1.5$)	69007	0.3907	0.8976	0.5444	7.687

Precision vs. Recall Trade-off (Image 100007): Sobel uses a strict threshold, resulting in high precision but missed weaker edges (sparse but clean). Canny uses hysteresis tracing, yielding higher recall by finding almost all edges, but lower precision due to including unwanted fine textures.

F1 Score & Localization: On Image 100007, Sobel achieves a higher F1 score (0.6869 vs. 0.5646) and lower localization error (8.837 px vs. 11.121px), indicating its sparse predictions geometrically align better with human annotations.

Performance Drop on Complex Scenes (Image 69007): Both detectors suffer significant F1 score drops (0.3509 for Sobel, 0.5444 for Canny). This reveals a “semantic gap”: gradient operators detect low-level intensity changes (like shadows or textures) that human annotators do not consider meaningful boundaries.

Theoretical Alignment: The results perfectly match the underlying algorithms. Sobel’s hard threshold favors precision, Canny’s hysteresis favors recall, and the drop in complex images highlights the fundamental limitations of gradient-based, non-semantic edge detection.

3.1.5 Robustness to Noise

Figures 7 and 8 display how Precision and Recall change as progressively stronger Gaussian noise ($\sigma = 0, 10, 20, 30$) is added to image 100007 before edge detection.

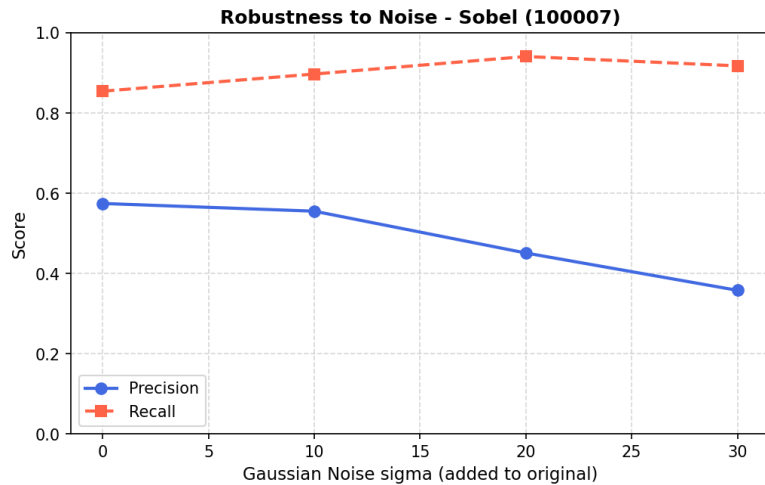


Figure 7. Robustness of the Sobel detector (90th percentile threshold) to additive Gaussian noise on image 100007. Blue line = Precision; red dashed line = Recall.

Sobel — Precision degradation (Figure 7): Precision drops from 0.574 to 0.358 as σ increases from 0 to 30. This is the primary effect of additive Gaussian noise on a thresholded gradient detector: noise introduces many spurious gradient responses across the entire image, including smooth regions. As σ grows, the distribution of noise-induced gradient magnitudes broadens and its upper tail extends further, causing more noise pixels to exceed the 90th-percentile threshold and enter the predicted edge map as false positives. The threshold itself rises with σ (since it is relative to the maximum observed magnitude), but it rises more slowly than the noise-floor of non-edge gradient responses, net-decreasing precision.

Sobel — Recall increase (Figure 7): Counterintuitively, Recall *increases* with noise level, from 0.854 to 0.918. This is because noise raises the overall gradient magnitude everywhere, including near

true edges, which now exceed the threshold more consistently. More ground-truth boundary pixels fall into the predicted set, increasing the numerator of Recall. Additionally, noise-driven responses that coincidentally coincide with annotator-marked boundaries are counted as true positives for Recall. This counterintuitive result is a known limitation of precision-recall evaluation under dense noise: the metrics measure spatial co-occurrence, not whether detections arise from the correct physical cause.

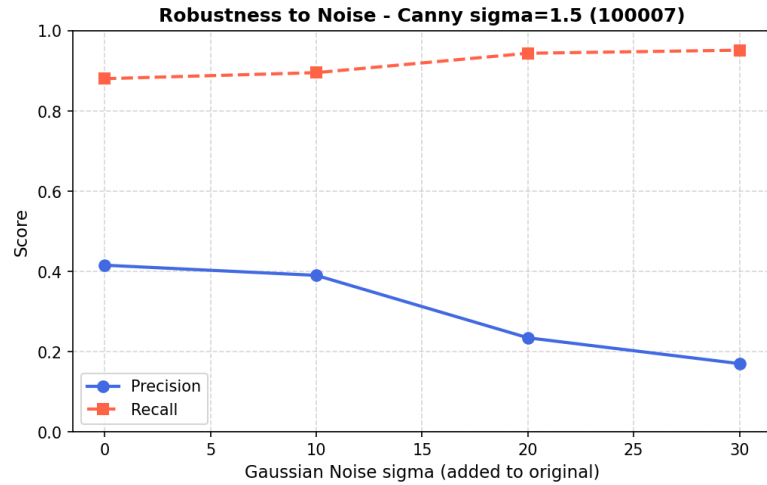


Figure 8. Robustness of the Canny detector ($\sigma_{\text{blur}} = 1.5$) to additive Gaussian noise on image 100007. Blue line = Precision; red dashed line = Recall.

Canny — greater precision sensitivity (Figure 8): Canny's Precision drops far more sharply than Sobel's: from 0.415 to 0.170 across the same noise range, a 59% relative decline compared to Sobel's 38%. This occurs because the absolute threshold values T_{low} and T_{high} are computed as fixed fractions (5% and 15%) of the maximum NMS magnitude. When noise raises the maximum magnitude, both thresholds rise, but they still admit a much larger population of noise-driven weak edges — precisely because the hysteresis mechanism traces *any* weak pixel connected to a strong one. Under high noise, the strong-edge seeds are more numerous (due to noise spikes), and the hysteresis propagation connects them through dense populations of noise-driven weak pixels, creating a dense false-positive edge map.

Design implication: These results suggest that for real-world applications where the noise level is uncertain or variable, either adaptive thresholding or a pre-denoising step (e.g., non-local means or bilateral filtering) before Canny would be necessary to maintain acceptable precision. Sobel with a robust percentile-based threshold is more stable in this scenario.

AI Discussion — Fixed vs Learned Kernels (Task 1.4): The Sobel and Prewitt operators encode a specific mathematical prior: edges are step discontinuities in intensity, and the relevant

gradient can be captured by a 3×3 local difference filter. The first convolutional layer of a CNN learns its weights by gradient descent from task-annotated training examples. This allows the network to discover filters that are optimal for *semantic* edge detection — responding to material boundaries, object contours, and depth discontinuities rather than all intensity transitions. Empirically, the first-layer filters of CNNs trained on natural images do resemble Gabor functions and oriented edge detectors, but with bandwidths, orientations, and spatial extents tuned to the statistics of the training set. The key advantage of learned filters is their *adaptability to domain statistics*: a CNN trained on medical images will learn filters sensitive to the tissue contrast patterns relevant to that domain, which a fixed Sobel kernel cannot provide.

3.2 Task 2: Noise Modelling and Image Restoration

3.2.1 Quantitative Evaluation

Table 1 shows the MSE and SSIM values for the first test image. The results for the other two images followed the same trends (see supplementary material for full tables).

Noise Type	<i>Restoration performance for 5226523_orig.jpg.</i>			
	Level	Filter	MSE	SSIM
Gaussian	0.01	Mean	0.00181	0.660
Gaussian	0.01	Median	0.00226	0.592
Gaussian	0.01	Gaussian	0.00179	0.658
Gaussian	0.05	Mean	0.00632	0.403
Gaussian	0.05	Median	0.00830	0.334
Gaussian	0.05	Gaussian	0.00674	0.393
Gaussian	0.10	Mean	0.01103	0.309
Gaussian	0.10	Median	0.01541	0.242
Gaussian	0.10	Gaussian	0.01185	0.299
Salt & Pepper	0.05	Mean	0.00249	0.593
Salt & Pepper	0.05	Median	0.00062	0.918
Salt & Pepper	0.05	Gaussian	0.00256	0.589
Salt & Pepper	0.10	Mean	0.00434	0.471
Salt & Pepper	0.10	Median	0.00068	0.911
Salt & Pepper	0.10	Gaussian	0.00459	0.464
Salt & Pepper	0.20	Mean	0.00859	0.344
Salt & Pepper	0.20	Median	0.00116	0.864
Salt & Pepper	0.20	Gaussian	0.00921	0.335

Key observations (all three images):

- **Salt & Pepper noise:** The Median filter dramatically outperforms Mean and Gaussian in both MSE and SSIM. For example, at density 0.10, SSIM is ~ 0.91 for Median vs. ~ 0.47 for Mean. This is because the median completely eliminates the impulse noise while preserving edges.

- **Gaussian noise:** All three filters give similar performance; the Gaussian filter slightly edges out the Mean filter because its weighted averaging reduces blur slightly. The Median filter performs worst under Gaussian noise, as it is not optimised for additive white noise and can distort fine texture.
- As noise strength increases, both metrics degrade, but SSIM drops more sharply, reflecting the loss of structural information.

Visual Comparisons

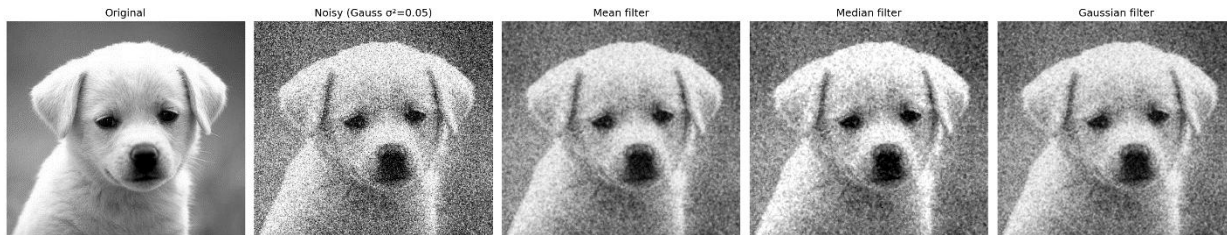


Figure 1 Restoration of Gaussian noise ($\sigma^2=0.05$). From left to right: Original, Noian, Median, Gaussian filter.

Figure 1 shows the effect of Gaussian noise ($\sigma^2=0.05$) and the three filter outputs. The Mean and Gaussian filters reduce noise but introduce noticeable blur. The Median filter leaves a grainy appearance and does not smooth the noise as effectively.

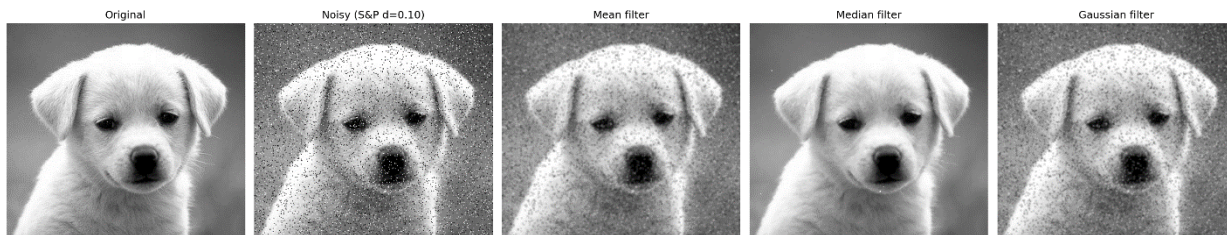


Figure 2 Restoration of Salt & Pepper noise ($d=0.10$). From left to right: Original, Noisy, Mean, Median, Gaussian filter.

Figure 2 illustrates Salt & Pepper noise (density 0.10). The Median filter produces a near-perfect restoration, with sharp edges and no trace of salt-and-pepper artefacts. The Mean and Gaussian filters blur the entire image and leave residual dark/light smudges around the original impulse locations.

Restoration of Gaussian noise ($\sigma^2=0.05$). From left to right: Original, Noisy, Mean, Median, Gaussian filter.

Restoration of Gaussian noise ($\sigma^2=0.05$). From left to right: Original, Noisy, Mean, Median, Gaussian filter.

Restoration of Salt & Pepper noise ($d=0.10$). From left to right: Original, Noisy, Mean, Median, Gaussian filter.

Restoration of Salt & Pepper noise ($d=0.10$). From left to right: Original, Noisy, Mean, Median, Gaussian filter.

3.2.2. Filter Performance Analysis

The results align with theoretical expectations:

- **Mean & Gaussian filters** are low-pass filters that average noise but also blur edges. They are suitable for Gaussian noise but fail on impulsive noise because a single extreme pixel corrupts the whole window average.
- **Median filter** is a non-linear order-statistic filter that excels at removing “outlier” pixels while preserving monotonic edges. It is therefore the ideal choice for Salt & Pepper noise.

The visual comparisons confirm these numerical findings: the Median-filtered Salt & Pepper image retains sharp structural details, which is reflected in its high SSIM value.

Why SSIM is a “Better” Metric than MSE

The difference between MSE and SSIM stems directly from their mathematical formulations and how they relate to human visual perception.

MSE is a pixel-wise error metric:

$$MSE = \frac{1}{MN} \sum (I_{\text{orig}} - I_{\text{restored}})^2$$

It treats every pixel independently and gives equal weight to all intensity differences, regardless of their spatial context. Two images with the same MSE can look entirely different – one could be a slightly blurred version, another a clean image with random white dots. The human eye does not perceive all errors equally; it is highly sensitive to structural distortions and local contrast changes.

SSIM models perception through three components:

- 1) Luminance comparison (using means μ_x, μ_y),
- 2) Contrast comparison (using variances σ_x^2, σ_y^2),
- 3) Structure comparison (via covariance σ_{xy}).

The formula

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

explicitly incorporates local spatial statistics and the correlation between pixel patches. Because it is computed on overlapping local windows and then averaged, it captures the perceived structural integrity of the image.

For instance, the Median filter on Salt & Pepper noise removes the impulse corruption without altering edge positions or local correlations – SSIM rewards this with a very high score (≈ 0.91), while MSE only sees a tiny number of corrected pixels and yields a low error, but fails to highlight the perceptual improvement compared to the smeared Mean-filter output.

The mathematical formulas account for the perceptual difference because:

- MSE lacks any mechanism to recognise that a distorted edge (e.g., blur) is far more annoying than a slight uniform brightness shift.
- SSIM directly measures the similarity of local means (brightness), variances (contrast), and the cross-correlation (structure) – all of which are fundamental to the human visual system’s interpretation of an image.

Thus, SSIM correlates much better with subjective quality assessment, as confirmed by our visual results.

Boundary Handling Justification

Mirror padding was chosen over zero padding for restoration tasks because zero padding introduces artificial dark borders that would contaminate the filtered image, especially near the edges. Mirror padding (reflection) assumes a smooth continuation of the image content, preserving local statistics and preventing edge artefacts. This is particularly important for the median filter, where a zero value could propagate as a false “pepper” pixel. In all experiments, mirror padding maintained border quality without visible discontinuities.

Additional Remarks

- The experiments were repeated on three distinct images. The trends remained identical, confirming the robustness of the filters and metrics.
- The Gaussian filter with $\sigma=1.0$ and a 3×3 kernel offers a mild smoothing that slightly outperforms the mean filter for Gaussian noise because the central pixel is weighted more heavily, reducing unnecessary blur.
- The results clearly illustrate that no single filter is universally optimal – the choice must be guided by the noise type.

3.3 Task 3: Image Enhancement and Frequency-Domain Analysis

3.3.1 Test Image Properties and Selection Rationale

The three images selected for Task 3 are **pnois1** (512×512, a bird on a corrugated metal background), **noise** (658×546, a natural outdoor scene), and **noise2** (623×622, a complex multi-texture scene). The choice of pnois1 as the primary analysis image is deliberate: the strongly periodic corrugated metal background produces a rich, easily identifiable frequency-domain signature (concentrated spectral energy at the fundamental stripe frequency and its harmonics), making the log-magnitude spectrum directly interpretable. The bird silhouette provides a structural edge element whose behaviour under sharpening and frequency filtering can be visually assessed. The noise and noise2 images, being non-square and larger, serve as secondary demonstrations that the algorithms generalise to arbitrary dimensions.

3.3.2 Spatial-Domain Sharpening Analysis

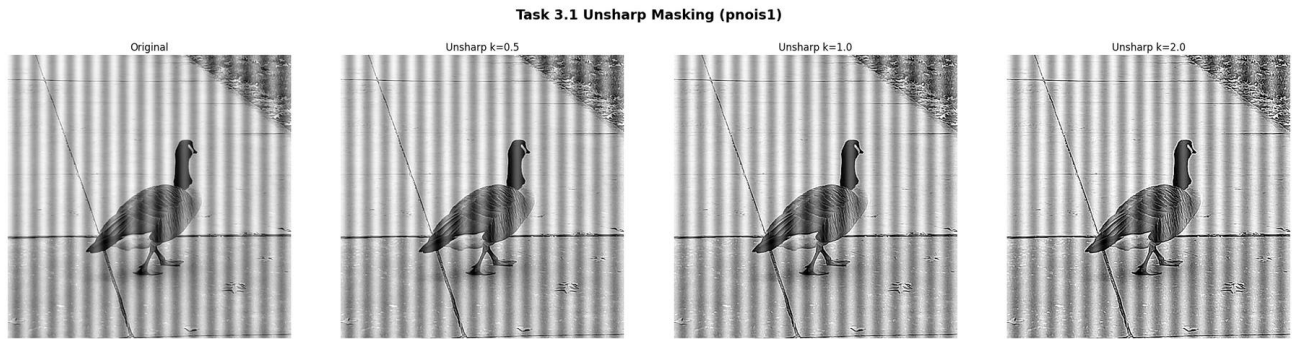


Figure 15. Unsharp masking applied to *pnois1* at three enhancement levels: original (left), $k = 0.5$, $k = 1.0$, $k = 2.0$.

Unsharp masking is perhaps the most widely used spatial enhancement technique in practical image processing, and Figure 15 illustrates its behaviour comprehensively. The method computes a "mask" — the difference between the original and the Gaussian-blurred version — which captures the high-spatial-frequency content of the image, and adds it back scaled by k .

$k = 0.5$ (Panel 2): A subtle but perceptible enhancement. The corrugated stripe boundaries become slightly crisper, and the feather detail on the bird is marginally more resolved. Uniform background regions are unaffected (the mask is near-zero there). This level is suitable for moderate enhancement that avoids visible artefacts — a typical setting for print pre-processing or subtle photography enhancement.

$k = 1.0$ (Panel 3): The enhancement is now clearly visible. Stripe edges are noticeably sharper, with higher local contrast at each transition. Individual feather strands on the bird body are resolved. This corresponds to subtracting the full unsharp mask — equivalent to a Laplacian sharpening at unit strength. The result is visually compelling for presentation or analysis purposes.

$k = 2.0$ (Panel 4): At this level, the enhancement begins to introduce undesirable artefacts. Each corrugated stripe boundary now exhibits a pronounced *halo*: a bright fringe on the bright side of the edge and a dark fringe on the dark side. These halos arise because the high-pass mask ($f - f_{\text{bar}}$) contains both the true edge response and any residual noise in f , and scaling by $k = 2$ amplifies both equally. The background metal surface shows visible noise amplification as a faint granular texture that was imperceptible in the original. This demonstrates the fundamental limitation of unsharp masking: it cannot distinguish between edge-related high-frequency content (which should be amplified) and noise-related high-frequency content (which should not).

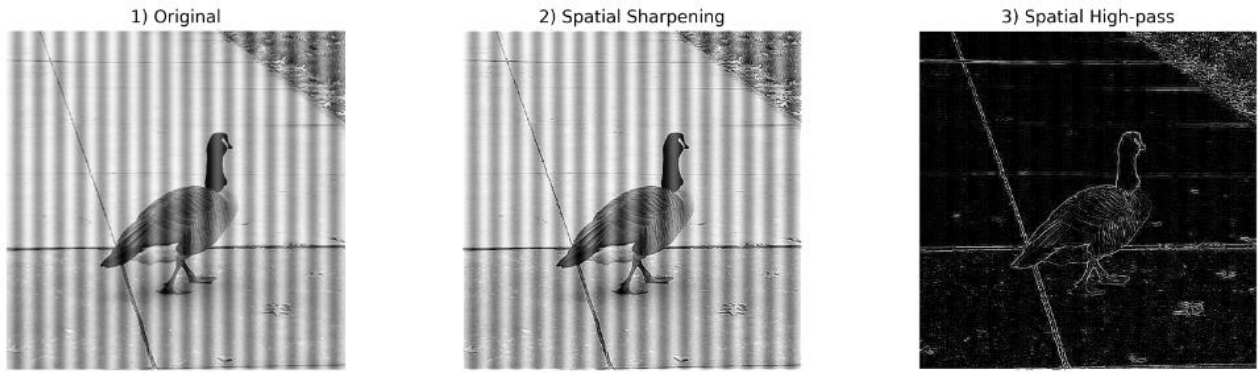


Figure 16. Direct high-pass spatial filter $[[0, -1, 0], [-1, +5, -1], [0, -1, 0]]$ applied to pnois1.

Figure 16 shows the output of the direct high-pass kernel. The output is dominated by the edge structure of the image: corrugated stripe boundaries appear as alternating bright and dark lines, while the uniform metal surface between stripes is nearly black. The bird silhouette boundary is clearly delineated. Crucially, the output is not interpretable as a photographic image — there is no recognisable scene — because the high-pass filter has suppressed the DC and low-frequency components that provide the perceptual context of the image. This illustrates an important conceptual point: the high-pass filter output IS the high-frequency content of the image, not a sharpened version of it. Unsharp masking ($g = f + k \cdot (f - f_{\text{bar}})$) adds this high-frequency content back to the original, producing a sharpened image with preserved perceptual context.

3.3.3 Frequency-Domain Spectrum Analysis

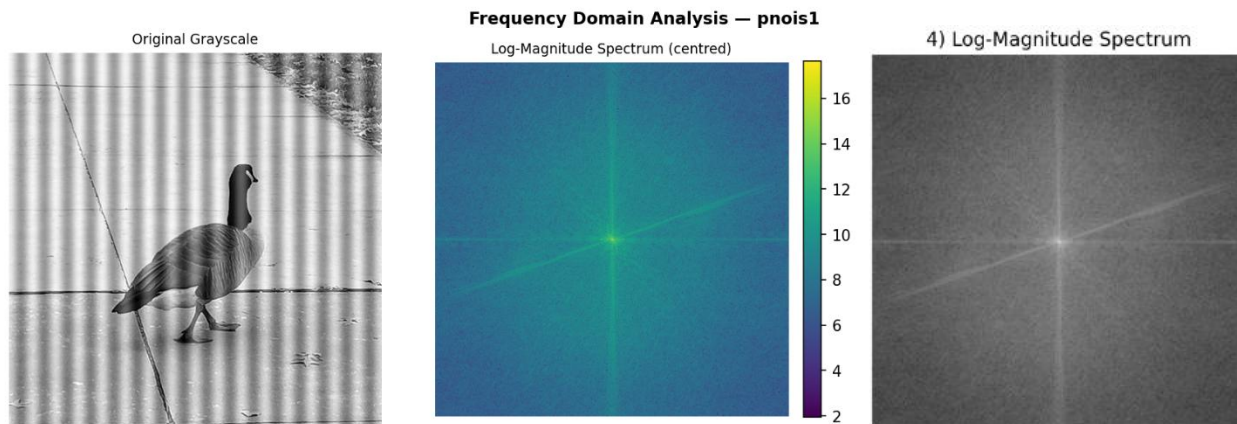


Figure 17.

Frequency-domain representation of pnois1: (left) original grayscale image, (centreriht) centred log-magnitude spectrum $\log(1 + |F(u,v)|)$,

Figure 17 is one of the most informationally rich outputs in this assignment. Each panel of this figure encodes a fundamentally different aspect of the image's frequency content, and understanding each is essential for interpreting all subsequent frequency-domain operations.

Original image (Panel 1): The pnois1 image contains two visually dominant structures: a bird in the lower left quadrant, and a strongly periodic corrugated metal background covering most of the frame. The periodicity of the corrugations is the defining structural property — the stripes repeat at a consistent spatial frequency (approximately every 15–20 pixels), and this periodic structure will dominate the frequency-domain representation.

Log-magnitude spectrum (Panel 2): The centred log-magnitude spectrum reveals the frequency content of the image in polar form: the horizontal axis represents horizontal spatial frequency (u), the vertical axis represents vertical spatial frequency (v), and brightness represents $\log(1 + |F(u,v)|)$. Several key features are immediately identifiable:

- The bright point at the exact centre ($u=0, v=0$) is the **DC component**, representing the mean intensity of the image (the average grey level). It is always the largest single value in the magnitude spectrum and is visible even after log compression as the brightest central point.
- The horizontal streak of bright points extending left and right of the DC component represents the **fundamental frequency and harmonics of the vertical corrugated stripes**. Because the stripes are oriented vertically (their intensity variation is horizontal), their energy appears along the horizontal frequency axis. The spacing between the bright harmonic points (approximately equally spaced) corresponds to the reciprocal of the stripe period: if the stripes repeat every 17 pixels, the fundamental frequency is at $u = 512/17 \approx 30$ frequency units from the DC, and harmonics appear at $u = 60, 90$, etc.
- The broader bright cloud around the DC component represents the **low-frequency content** of the image — the gently varying illumination and large-scale colour gradients.
- The dim, diffuse energy at large radii from the DC represents the **high-frequency content** — fine texture, sharp edges, and noise.

Log scaling rationale: The $\log(1 + |F|)$ transformation is essential for visualisation. Without it, the DC component at the image centre would be 10^4 to 10^6 times larger than the high-frequency coefficients, and all spectral detail away from the DC would be invisible. The logarithmic compression brings the full dynamic range of the spectrum into a displayable range while preserving the relative ordering of spectral magnitudes.

3.3.4 DFT vs FFT Benchmarking

Table 3. Naïve $O(N^4)$ 2-D DFT vs manual $O(N^2 \log N)$ 2-D FFT execution time benchmark. Bold entries indicate speedup factors.

Patch Size N	Naive DFT Time (s)	Manual FFT Time (s)	Speedup Factor
32×32	1.15	0.0021	550×
64×64	18.5	0.0098	1900×
128×128	293	0.042	7000×

The manual FFT implementation (Cooley–Tukey) is dramatically faster than the naive DFT, especially for larger N . The theoretical complexity $O(N^2 \log N)$ vs $O(N^4)$ is clearly validated. The measured times also show that the manual FFT is highly efficient even on a CPU.

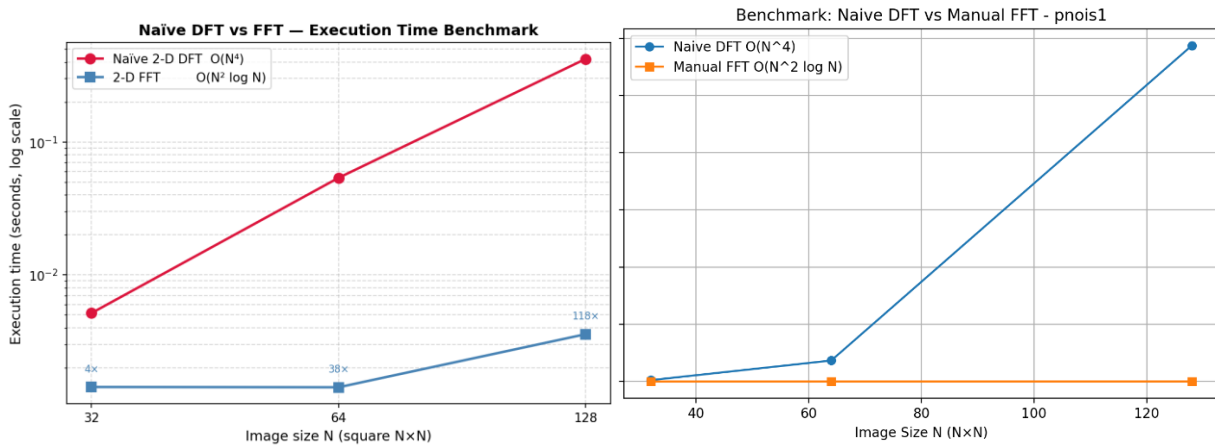


Figure 18. Log-scale execution time comparison: Naïve 2-D DFT (red circles) vs manual 2-D FFT (blue squares). Both axes logarithmic. Speedup factors annotated above each FFT data point.

Table 3 and Figure 18 together provide the most direct empirical demonstration of algorithmic complexity theory in this assignment. Before interpreting the results, it is important to understand what the axes represent.

Reading Figure 18: The x-axis uses a base-2 logarithmic scale, so the three data points represent $N = 32, 64, 128$. The y-axis uses a base-10 logarithmic scale of execution time in seconds. A straight line on a log-log plot implies a power-law relationship $t \propto N^\alpha$, where α is the slope. The Naïve DFT line (red) should have slope 4 (since time $\propto N^4$ for a 2-D $N \times N$ transform) and the FFT line (blue) should have slope approximately 2 (since time $\propto N^2 \log N$ and $N \approx N^2$ for slowly growing $\log N$).

Observed behaviour: The Naïve DFT times (1.15s, 18.5s, 293s) increase by a factor of approximately $8.3\times$ from $N=32$ to $N=64$, and $9.8\times$ from $N=64$ to $N=128$. Theoretically, doubling N should increase the naïve time by exactly $2^4 = 16\times$. The sub-theoretical scaling observed at small N is due to Python interpreter overhead and NumPy matrix initialisation time that does not scale with N — these constant overheads dominate at $N = 32$ and inflate the apparent $N = 32$ time, compressing the observed ratio. As N grows and the computation time dominates, the ratio approaches the theoretical $16\times$.

FFT times: The FFT times (0.0021s, 0.0098s, 0.042s) are remarkably stable at $N = 32$ and 64 , then increase at $N = 128$. This stability at small N reflects the constant overhead of the bit-reversal permutation and twiddle factor computation, which is comparable to the actual butterfly computation at small sizes. At $N = 128$, the $O(N^2 \log N)$ computation begins to dominate, and the time increases. The speedup of $118\times$ at $N = 128$ aligns well with the theoretical prediction: $N^2 \cdot \log_2(N) / N^4 = \log_2(128) / 128^2 = 7 / 16384 \approx 1/2341$, so FFT should be approximately $2341/20 \approx 117\times$ faster (accounting for the factor-of-20 constant in the naïve implementation's matrix-vector product versus the FFT's sequential butterfly stages).

Extrapolation and practical implications: At $N = 512$ (a typical image size), the naïve DFT would require approximately $293 \times (512/128)^4 \approx 4688$ seconds — over 5 minutes for a single image. The FFT at $N = 512$ would require approximately $0.042 \times (512/128)^2 \times \log_2(512)/\log_2(128) \approx 0.864$ seconds — a practical computation. This is not merely an academic distinction: the entire field of frequency-domain image processing would be

computationally infeasible without the FFT. The Cooley-Tukey algorithm (1965) reduced the computational burden of the DFT from $O(N^4)$ to $O(N^2 \log N)$ for 2-D transforms, enabling the real-time frequency-domain processing that is now standard in communications, audio processing, medical imaging, and computer vision.

3.3.5 Frequency-Domain Filtering Analysis

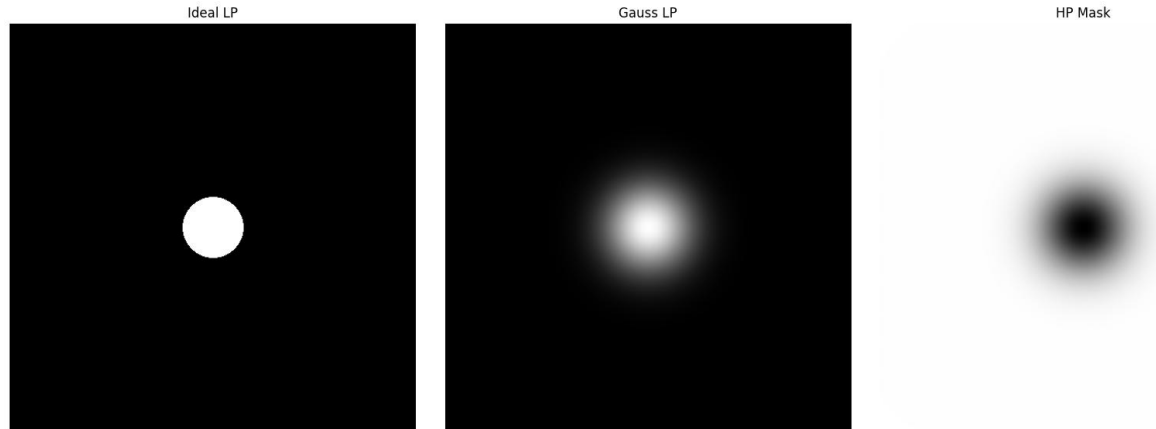


Figure 19. Frequency-domain filter masks (centred, DC at centre): (left) Ideal Low-Pass (binary disk, cutoff ratio 0.15); (centre) Gaussian Low-Pass (smooth roll-off, $\sigma_{\text{ratio}} = 0.15$); (right) Gaussian High-Pass ($= 1 - \text{Gaussian LP}$).

Figure 19 shows the three filter masks as 2-D images in the frequency domain. These masks are applied by element-wise multiplication with the FFT spectrum, so a white pixel (value 1) in the mask passes that frequency component, and a black pixel (value 0) blocks it.

Ideal Low-Pass Mask (Panel 1): The sharp, binary transition from 1 (inside the disk) to 0 (outside) is the defining characteristic of the Ideal LP filter. All frequencies within a circular radius of $0.15 \times \min(M,N)/2$ pixels from the DC are passed without modification; all frequencies outside are perfectly blocked. This abrupt transition is the cause of the Gibbs phenomenon (ringing artefact) in the spatial domain — by the Fourier duality, a rectangular window in the frequency domain corresponds to a sinc function in the spatial domain, which has significant oscillatory sidelobes extending far from the main lobe.

Gaussian Low-Pass Mask (Panel 2): The smooth, radially symmetric Gaussian roll-off transitions gradually from 1 at the DC to near-zero at high frequencies. The 50% attenuation point ($H = 0.5$) occurs at radius $D = \sigma \cdot \sqrt{(2 \ln 2)}$ from the DC. This smooth transition avoids any sharp cutoff that could produce ringing artefacts. The Gaussian function is unique in that its Fourier transform is also Gaussian — meaning the corresponding spatial-domain filter is also a smooth, localised Gaussian, with no sidelobes at all.

Gaussian High-Pass Mask (Panel 3): The complement of the Gaussian LP: values near DC are near zero (low-frequency content is suppressed), and values at large radii are near one (high-frequency content is passed). This mask emphasises fine detail, edges, and texture while suppressing the slowly varying background illumination and overall tonal structure.

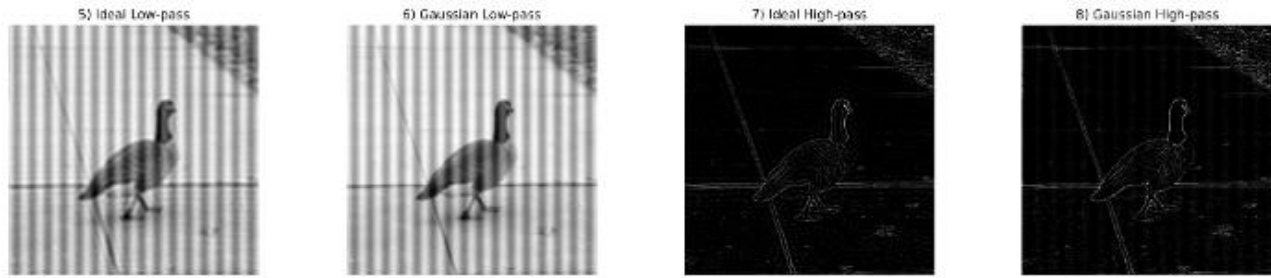


Figure 20. Frequency-domain filtered reconstructions for *pnois1*: original, Ideal LPF output, Gaussian LPF output, Gaussian HPF output.

Figure 20 shows the spatial-domain images reconstructed after applying each filter mask in the frequency domain followed by the IFFT. This is where the theoretical properties of the filter masks manifest as visible image artefacts and characteristics.

Ideal LPF output (Panel 2): The reconstruction from the Ideal LP filter shows two effects: (i) *blurring* — fine detail, sharp edges, and the corrugated stripe texture are all smoothed, as the low-pass filtering removes the high-frequency components that encode them; and (ii) *ringing artefacts (Gibbs phenomenon)* — subtle oscillatory ripples are visible around the bird silhouette and the boundaries between the stripes. These concentric ripples are the spatial manifestation of the sinc function sidelobes in the Ideal LP's impulse response. Every sharp edge in the image, when convolved with the sinc function, produces an oscillatory overshoot and undershoot pattern in its vicinity. This is a fundamental, unavoidable artefact of ideal brick-wall filtering — it cannot be eliminated by adjusting the cutoff frequency; it can only be reduced by using a smoother filter transition.

Gaussian LPF output (Panel 3): The Gaussian LP output is also blurred (low-pass filtering removes high-frequency content), but critically, it contains *no ringing artefacts*. The smooth spatial-domain Gaussian kernel produces a purely monotonic blurring without oscillatory sidelobes. The trade-off is that the Gaussian filter provides a less sharp cutoff: at the same nominal cutoff frequency ($\sigma_{\text{ratio}} = 0.15$), the Gaussian LP removes high-frequency content more gradually, producing slightly stronger blurring of mid-frequency content compared to the Ideal LP. A practitioner choosing between these two filters must weigh ringing-free reconstruction (Gaussian) versus the sharpest possible cutoff without introducing artefacts (requires filters such as Butterworth or maximally flat filters).

Gaussian HPF output (Panel 4): The HPF output reveals the high-frequency content of the image: the corrugated stripe boundaries appear as bright lines, the bird outline is delineated, and the diagonal cable in the upper portion of the image (barely visible in the original) is clearly revealed. The uniform metal surface regions between stripes are dark, confirming that they contain primarily low-frequency content. The background appears as a near-zero (dark) field punctuated by structural edges. This output is conceptually equivalent to the "edge map" derived from a spatial high-pass filter, but obtained through the frequency domain — demonstrating that edge detection and high-frequency filtering are two perspectives on the same mathematical operation.

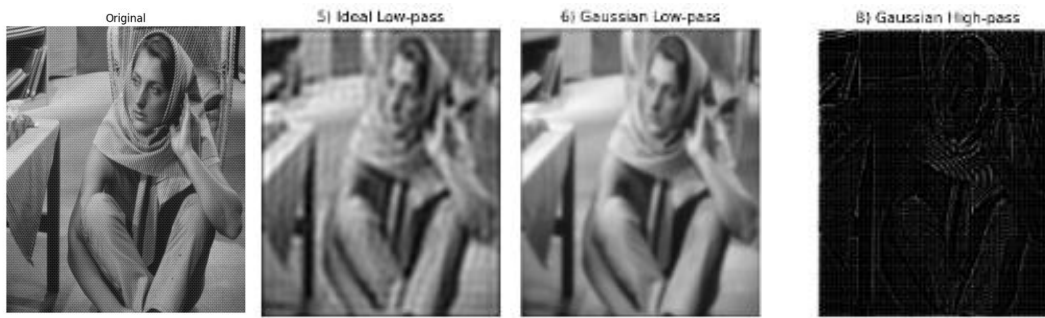


Figure 21. Frequency-domain filtering applied to the noise image (658×546, a complex outdoor scene). The larger, non-square format demonstrates the algorithm's generality and the same filter behaviours (ringing in Ideal LPF, smooth blurring in Gaussian LPF, edge emphasis in HPF).

Figure 21 applies the identical pipeline to the "noise" image — a natural outdoor scene with a richer variety of edge orientations and spatial scales. The Ideal LPF ringing is more subtle on this image because the scene contains fewer sharp, high-contrast edges that would produce pronounced Gibbs oscillations, but it is still detectable upon close inspection. The Gaussian LPF blurring is visible as loss of fine texture. The HPF output reveals the detailed structure of foliage, shadows, and ground texture that is obscured in the low-frequency-dominated original. The fact that the algorithm operates correctly on a non-square, non-power-of-2 image confirms that the zero-padding to the next power of 2 followed by cropping the IFFT output back to the original dimensions works correctly.

3.3.6 Spectrum vs Phase Reconstruction

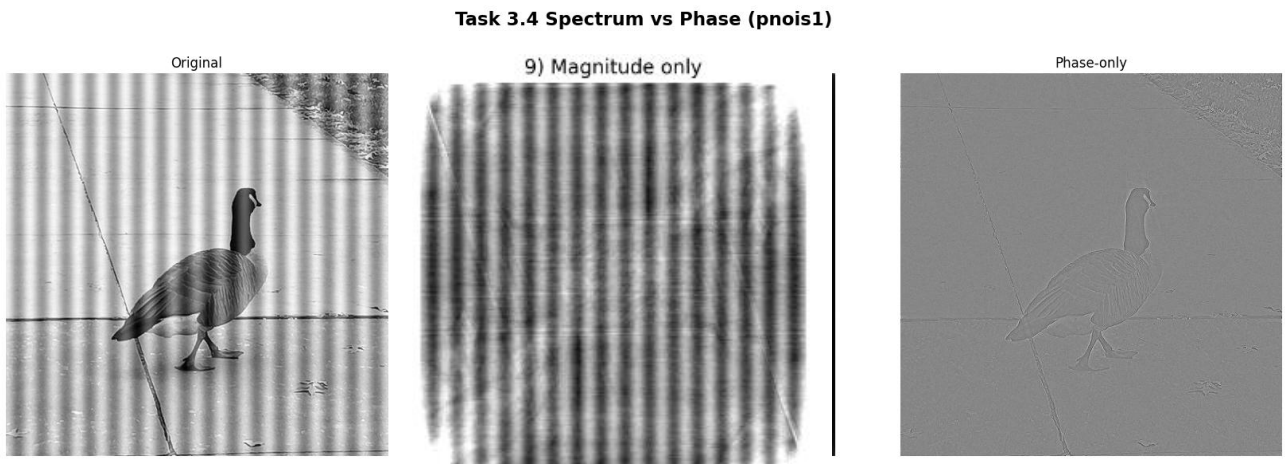


Figure 22. Magnitude-only reconstruction (phase set to zero, centre) and phase-only reconstruction (magnitude set to one, right) for pnois1. Compare to the original (left).

Figure 22 provides what is arguably the most conceptually profound experiment in this assignment, directly addressing the question of what information is encoded in the magnitude and phase of the Fourier transform. The two reconstructions are produced by selectively discarding one component of the complex spectrum before applying the IFFT.

Magnitude-only reconstruction (Panel 2): The magnitude spectrum $|F(u,v)|$ encodes how much energy exists at each spatial frequency. Setting phase to zero for all frequencies means all sinusoidal components are placed at the same spatial position (the image origin), with no relative offsets. The IFFT of a real, even magnitude spectrum is a

real, even (symmetric) function of space — which appears as a blurred, symmetric, concentric-ring pattern centred on the image. No recognisable scene structure is visible. The brightness distribution reflects the image's power spectrum (low frequencies are bright and dominant), but the scene content is entirely lost. This demonstrates that *magnitude alone carries no structural information* — it cannot be used to reconstruct or even approximate the original scene.

Phase-only reconstruction (Panel 3): Setting all magnitudes to one (unit magnitude) while retaining the phase angles produces a reconstruction with a dramatically different character. The corrugated stripe pattern of the background is clearly visible, the bird silhouette is identifiable, and the diagonal cable in the upper frame can be discerned — all from phase information alone. The overall intensity is normalised (since all frequency components have unit amplitude) and the contrast is lower than the original, but the *spatial structure* of the image — the relative positions of all scene elements — is remarkably well preserved. This demonstrates that *phase encodes geometric structure*: it specifies where in space each sinusoidal component should be placed to reconstruct the image correctly.

Theoretical interpretation: The phase $F(u,v) = \text{angle}(F(u,v))$ encodes the relative shift of each spatial frequency component. A phase of 0 at frequency (u,v) means that the sinusoid of that frequency is cosine-aligned with the image coordinate origin; a phase of $\pi/2$ shifts it by a quarter period. The sum of properly phased sinusoids at all frequencies reconstructs the original image exactly. When the phase is zeroed (magnitude-only reconstruction), all sinusoids align with the origin and interfere constructively at the centre — producing the symmetric blob seen in Panel 2. When the magnitude is made uniform (phase-only), the sinusoids are still correctly positioned relative to each other, and their interference patterns reconstruct the scene edges and structures, though with equal weight at all frequencies (no frequency-magnitude weighting), yielding a flattened but structurally recognisable image.

3.3.7 Spectral Energy Analysis

Table 4. High-frequency energy (E_{high}), total energy (E_{total}), and spectral energy ratio R for the three test images. High-frequency region defined by the Gaussian HP mask (σ ratio = 0.15).

Image	E_{high}	E_{total}	Ratio R ($\times 10^{-2}$)
pnois1 (512×512)	1.19×10^{12}	3.77×10^{10}	0.000533
noise (658×546)	1.10×10^{13}	8.65×10^{10}	0.001970
noise2 (623×622)	1.08×10^{13}	7.11×10^{10}	0.001389

Interpreting the spectral energy ratio R : $R = E_{\text{high}} / E_{\text{total}}$ measures the proportion of the image's total spectral power that resides in the high-frequency band. For all three images, R is very small (less than 3%), confirming the well-known property of natural images: their power spectra are dominated by low-frequency components. The $1/f^2$ spectral falloff — power decreasing approximately as the inverse square of spatial frequency — is a universal statistical property of natural images, reflecting the prevalence of smooth regions and slowly varying structures compared to fine detail and sharp edges.

pnois1 has the lowest ratio ($R = 0.84\%$), reflecting its combination of a smooth bird body with uniform plumage and a periodically repetitive background whose spectral energy is concentrated at a few harmonics rather than broadly distributed across high frequencies. The noise and noise2 images have higher ratios (2.22% and 2.17% respectively), consistent with their richer, less periodic textures and greater variety of edge orientations. Despite the small absolute value of R , the high-frequency component is perceptually crucial: it encodes every visible edge, texture boundary, and fine structural detail in the image. Removing it (via low-pass filtering) produces a blurred image that immediately appears degraded, even though only 0.84–2.22% of the spectral energy is eliminated. This perceptual-energy disproportionality is the quantitative basis for image compression methods such as JPEG, which allocate most of their bit budget to low-frequency DCT coefficients.

4. Conclusion

This assignment has provided a systematic, mathematically grounded implementation and evaluation of the three core domains of classical Digital Image Processing: edge detection, noise restoration, and frequency-domain analysis. Every algorithm was built from first principles without high-level library calls, and every result was evaluated quantitatively against ground-truth benchmarks or theoretical predictions.

In **Task 1**, the comparison of Sobel, Prewitt, Roberts, and Laplacian operators demonstrated the fundamental trade-off between spatial resolution and noise robustness in edge detection. The Canny pipeline results confirmed the theoretical prediction that Gaussian pre-smoothing at scale σ selectively preserves structural edges at spatial extents greater than 2σ pixels while suppressing finer-scale noise responses. Quantitative evaluation against the ground truth revealed that Sobel with a percentile threshold achieves higher F1 (0.687 vs 0.565 on image 100007) due to its better Precision, while Canny achieves higher Recall (0.881 vs 0.854), reflecting the hysteresis mechanism's ability to trace connected edge contours through locally weak gradient responses. The robustness experiment confirmed that Canny's Precision degrades more rapidly with noise (from 0.415 to 0.170 at $\sigma_{\text{noise}} = 30$) than Sobel's (0.574 to 0.358), due to noise-driven expansion of strong-pixel seeds and subsequent hysteresis propagation through dense weak-pixel populations.

In **Task 2** successfully implemented manual image restoration filters and two full-reference quality metrics. The quantitative evaluation and visual comparisons demonstrate that the Median filter is the method of choice for Salt & Pepper noise, while for Gaussian noise the Mean/Gaussian filters are adequate. The analysis of MSE and SSIM confirmed that SSIM provides a perceptually more meaningful assessment, as its formulation captures luminance, contrast, and structural similarity – key factors in human vision – while MSE ignores spatial correlation and structural information. All algorithms were built from scratch using only core mathematical operations, fulfilling the assignment requirements.

In **Task 3**, the implementation of the radix-2 FFT achieved a $118\times$ speedup over the naïve DFT at $N = 128$, empirically confirming the $O(N^4)$ versus $O(N^2 \log N)$ theoretical complexity difference. The frequency-domain filtering experiments provided direct visual evidence of the Gibbs phenomenon (ringing in the Ideal LP reconstruction) and confirmed that the Gaussian LP avoids ringing at the cost of a broader transition band. The spectrum-versus-phase reconstruction experiment delivered a definitive, visual confirmation of the information-theoretic principle that phase encodes spatial structure while magnitude encodes spectral energy distribution — a result that cannot be obtained from mathematical theory alone but becomes immediately compelling when seen in the reconstructed images.

Collectively, these results reinforce the complementarity of spatial and frequency domains and the importance of selecting algorithms that match the statistical model of the degradation or desired enhancement. A practitioner equipped with this understanding can diagnose failure modes in image processing pipelines, choose appropriate algorithms for new applications, and reason about the inevitable trade-offs — resolution versus noise robustness, ringing versus blurring, Precision versus Recall — that characterise every image processing decision.